AI534 Final exam review

Note that this review is not intended to be all inclusive, as we do not have sufficient time.

Rather this is an opportunity to review some of the key points

Coverage of the final exam

- Naïve Bayes classifier
- Decision trees
  - Entropy, Information gain, greedy algorithm for building decision trees
- Ensemble methods
  - Bagging
  - Random forest
  - Boosting
  - Bias variance decomposition
- Clustering
  - Hierarchical clustering: single, average, complete link
  - Kmeans clustering and variants
  - Mixture of Gaussian
  - Expectation maximization
- Dimension reduction methods
  - LDA
  - PCA
  - ISOMAP
  - t-SNE
- Neural networks
  - Representation power
  - Different activation functions
  - Training neural networks --- backpropagation
  - Making it work in practice: proper initialization, things that can help training, overfitting prevention etc.

Bayes Classifier

Learning steps:

- Learn $P(y)$
- Learn $P(\mathbf{x}|y=1), P(\mathbf{x}|y=2), \ldots, P(\mathbf{x}|y=K)$ each as a large joint distribution table with 2^d entries

MLE estimate of the joint distribution $P(\mathbf{x}|y=i)$:

Each entry gets a probability estimated as:

$$\frac{\#\text{ of class } i \text{ training examples with feature vector } \mathbf{x}}{\#\text{of class } i \text{ training examples}}$$

! Most entries will have $p=0$, severely overfits

Parameter size

$K \times (2^d - 1) + K - 1$

Naïve Bayes Classifier

Learning steps:

- Learn $P(y)$
- Learn $K \times d$ binary distributions:

$P(x_1|y=1), P(x_2|y=1), \ldots, P(x_d|y=1)$

$P(x_1|y=2), P(x_2|y=2), \ldots, P(x_d|y=2)$

$\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots$

$P(x_1|y=K), P(x_2|y=K), \ldots, P(x_d|y=K)$

MLE estimate of the distribution $P(x_j|y=i)$:

$$\frac{\#\text{ of class } i \text{ training examples with } x_j = 1}{\#\text{of class } i \text{ training examples}}$$

Naïve Bayes assumption

Features are **conditionally independent** given class label

Parameter size

$K \times d + K - 1$

Review the example

Apply Naïve Bayes to the following data and use the learned model to compute: $P(y = 1|X = (1,0,0))$

X_1	X_2	X_3	Y
1	1	1	0
1	1	0	0
0	0	0	0
0	1	0	1
1	0	1	1
0	1	1	1

$$P(y = 1|(1,0,0))$$
$$= \frac{p(y = 1)P(X = (1,0,0)|y = 1)}{P\big(X = (1,0,0)\big)}$$

$$= \frac{p(y = 1)P(x_1 = 1|y = 1)P(x_2 = 0|y = 1)P(x_3 = 1|y = 1)}{P\big(X = (1,0,0)\big)}$$

A common mistake

Computing the denominator of the Bayes Rule:

$$P\big(X = (1,0,0)\big) = P(x_1 = 1)P(x_2 = 0)P(x_3 = 0) = \frac{1}{2} * \frac{1}{3} * \frac{1}{2}$$

What is wrong with this ?

How should we calculate $P\big(X = (1,0,0)\big)$?

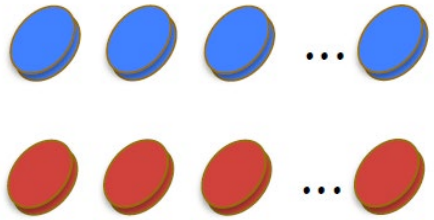
Multinomial model vs Bernoulli model

Bernoulli model:

Each feature $x_i \in \{0,1\}$

$$P(\mathbf{x}|y) = \prod_{i=1}^{V} P(x_i|y)$$

Each $P(x_i|y)$ is a Bernoulli distribution



Single occurrence is no different to multiple occurrences

Multinomial model:

Each feature $x_i \in N_0$ (i.e., nonnegative integer)

$$P(\mathbf{x}|y) = \prod_{i=1}^{V} P(w_i|y)^{x_i}$$

Multiple occurrences make a difference.
Allow us to take counts into consideration

Lesson learned from IA3

- Naïve Bayes learns a linear model
  - For multinomial naïve bayes, the linear coefficient for word i is:

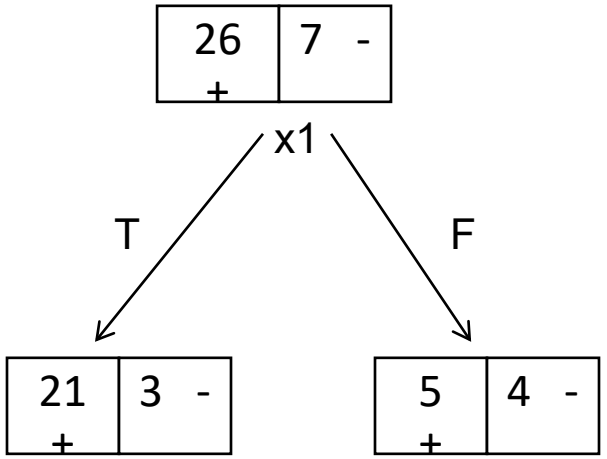
$$w_i = \log P(word_i|y = 1) - \log P(word_i|y = 0)$$

  - A similar formular was derived for Bernoulli as well
  - The bigger is the difference between P(word|y=1) and P(word|y=0) the larger the weight magnitude
- Impact of the smoothing parameter α (aka the pseudo count per word):
  - increasing α: $P(word|y)$ will be closer to uniform, the corresponding weights will be smaller → stronger regularization, less likely to overfit, more likely to underfit

Learning decision tree:

Greedily select the test that minimizes the expected uncertainty after the split

Using Entropy as the measure of uncertainty, we select x that maximizes $I(x, y)$, or equivalently minimize conditional entropy $H(y|x)$



Original entropy:

$$H(y) = .746$$

Left branch: $P(x_1 = T) = \frac{24}{33}$; $H(y|x_1 = T) = .544$

Right branch: $P(x_1 = F) = \frac{9}{33}$; $H(y|x_1 = F) = .991$

Conditional entropy:

$$H(y|x_1) = \frac{24}{33} * .544 + \frac{9}{33} * .991 = .6659$$

Properties of Entropy:

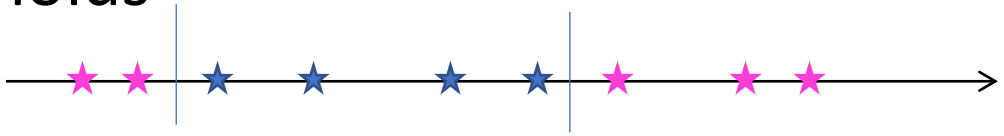
- Nonnegative: $H(X) \geq 0$
- Chain rule: $H(X, Y) = H(X|Y) + H(Y)$
- Monotonicity: $H(Y|X) \leq H(Y)$

Property of mutual information:

- Non-negative
- Symmetric

Based on the understanding of the decision tree algorithm, there are a variety of properties that follows

- Can a feature appear multiple times in a decision tree?
  - For binary feature, it can not appear multiple times on a **single path**, but can repeatedly appear on different paths
  - continuous feature can appear multiple times on the same path
- Multinomial Features with multiple splits will be favored by the selection criterion
- Working with continuous features only needs to consider class-switching thresholds



Decision tree overfits

- For any training set, we can always learn a decision tree to have 100% training accuracy
- One can stop early using some stopping condition or grow a large tree then prune it back
  - What are some advantages and disadvantages of both approaches?

Regression tree

- A regression tree partition the input space into axis parallel hyperrectangles and give each hyperrectangle a prediction of mean and variance (estimated based on training examples in that hyperrectangle)
- Learned with the same algorithm but use $Var(y)$ as the measure of uncertainty
- Can handle both discrete and continuous features without needing to do one hot encoding
- For continuous feature, need to consider all possible n-1 thresholds

Ensemble methods

Bagging	Random Forest	Boosting

• Can be used with any base classifier
• Parallelizable
• Sample examples w. replacement
• Majority voting (equal weights for all classifier)
 	
• Hyperparameters:
  • Ensemble size
  • Other parameters of the base classifier
	• Use decision tree as base classifier
	• Parallelizable
	• Sample examples w. replacement
	• For each test selection
	  • Sample m features to consider
	• Majority voting
	• Hyperparameters:
	  • Ensemble size
	  • Feature sample size: m, commonly set to be $\sqrt{d}$
	  • Other tree parameters to control the complexity of individual trees
		• Can be used with any base classifier
		• Iterative, not parallelizable
		• Each iteration, given the current distribution D_l:
		  • Learn a classifier h_l with D_l,
		  • Compute h_l 's weighed error ϵ_l on D_l
		  • Compute h_l 's voting weight based on ϵ_l
		  • Increase (decrease) weights of examples incorrectly (correctly) classified by h_l to create D_{l+1}
		  • After update/normalization, h_l's weighted error rate on D_{l+1} is 50%
		• Weighted voting
		• Hyperparameters:
		  • Ensemble size
		  • Other parameters of the base classifier

Properties

Bagging	Random Forest	Boosting
• Bagging achieves performance improvement primarily through reducing the variance • Increasing the ensemble size has little impact on overfitting • Due to the sampling operation, reduced sensitivity to outliers and noise	• Similar to bagging, RF reduces the variance • Individual trees in the forest can be less accurate than a regular decision tree as they introduce more randomness in the learning of the tree • Increasing ensemble size has little impact on overfitting • Reduced sensitivity to outliers and noise	• Can improve both bias and variance • Early iterations of the Boosting focus on reducing bias • Later iterations focuses on increasing the margin, and reducing the variance • Can be sensitive to outliers and noise • Weights of such examples can become exceedingly large • More sensitive over fitting as we increase ensemble size

Clustering

- Hierarchical Agglomerative clustering
  - Single-link, complete-link, average-link methods
  - Proper creation and interpretation of dendrogram
  - What types of clusters can each method find?

- Kmeans clustering
  - Why does it guarantee convergence? Each step reduces the objective while not converged
  - What kind of solution does it converge to? Local optimum, sensitive to initialization
  - What types of clusters can kmeans find?

- Gaussian mixture model (GMM) via Expectation Maximization
  - The E-step and M-step:
    - Correspondence between kmeans and EM under the shared spherical Gaussian assumption
    - you should be able to solve simple E-step and M-step like the one in the practice quiz and WA.
  - How do we know it converges? It performs Optimization transfer
  - What kind of solution it converges to? Local optimum, sensitive to initialization
  - What kind of clusters can Mixture of Gaussian find?

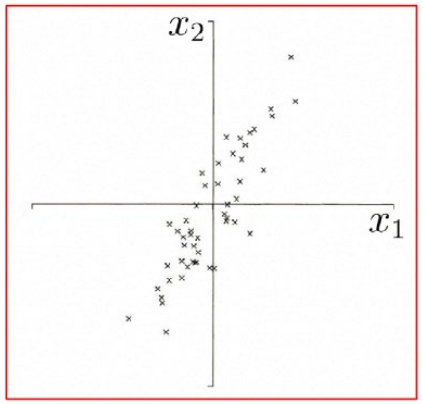
Selecting k and evaluating clustering solution

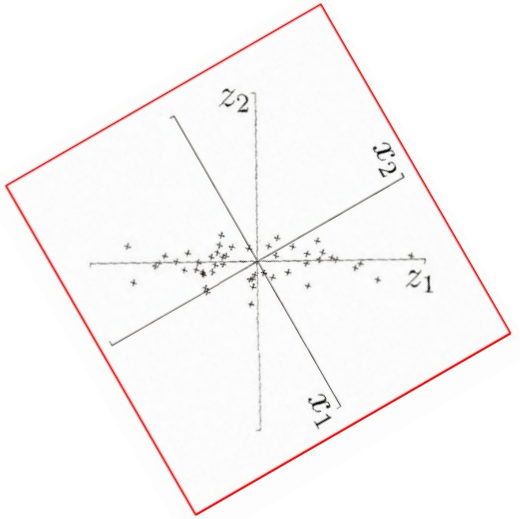
- How do we find clusters based on dendrogram?

- How does changing k influence the achievable objective of kmeans and GMM?
- What are some strategies for selecting k?
  - Look for the albow or knee
  - Use stability method: better k leads to more stable clustering solution
- External vs. internal evaluation criterion
- Various external criteria:
  - Adjusted rand index (why we need to use this as opposed to rand index?)
  - Purity
  - Normalized mutual information
  - Are they sensitive to the choice of k?

Dimension reduction: linear methods

- Linear discriminant analysis (supervised):
  - Objective: maximizing the separation of class mean & minimize the within-class spread
  - Why do we need both?
  - For binary class, we get only one dimension





- Principal component analysis (unsupervised)
  - Maximizing the variance after projection
  - Minimizing the reconstruction loss
  - Can select arbitrary number of dimensions to retain
  - When we do not reduce the dimension, PCA performs a rotation to the data to remove the correlation between the dime
  - When reducing dimension, discards low variance directions --- such information can be useful for classification. So applying PCA can discard useful information, thus hurting the downstream classification performance.

Dimension reduction: nonlinear methods

- ISOMAP
  - Construct a neighborhood graph,
  - Local neighborhood is Euclidean, thus we can directly apply Euclidean distance as edge weights
  - For further away points, compute distance using shortest path distance
  - Embed the data in low dimensional space such that the Euclidean distance approximate the geodesic distance
  - What can go wrong?
- t-SNE
  - Construct a similarity distribution based on the original dimensions using Gaussian kernel
  - Compute a similarity distribution in the embedded space using student-t distribution
  - Optimize the arrangement of the points in the embedding space to match the two distribution using K-L divergence
    - Implication of the asymmetry of k-l divergence
  - Impact of perplexity parameter (IA4)

Some concept questions for neural networks.

1. Why do we initialize the neural network with **small** and **random** weights? (what is wrong with large values? what is wrong with all the same value?)

2. What is the vanishing gradient issue? Why does ReLU activation help with this issue?

3. Why does using ReLU activation lead to sparse activation? What happens to the gradient when ReLu activation is turned off? What is the reason for using Leaky ReLU or ELU?

4. Why do we apply weight decay in training neural networks?

5. Given a Boolean function (in DNF form), you should be able to create a network representing this function

6. Why does the momentum method do for neural network training?
7. Why do we need to do early stopping in neural network training? How do we decide when to stop?